the keys start on January 1, 2005 (`"key_as_string" : "2005-01-01");` in the latter (`fixed_interval: 365d`), they start on the first release date, December 23, 2004 (`"key_as_string" : "2004-12-23"`). The second bucket adds 365 days from the first release date, yielding December 23, 2005 (`"key_as_string" : "2005-12-23"`).

> **NOTE** When we use `fixed_interval`, the range starts from the first document's available date. Going forward, the `fixed_interval` is added to that. For example, if the `publish_date` of a document is 25-12-2020 and we set the interval as `"month"`, the range starts with 25-12-2020 and goes to 25-01-2021, 25-02-2021, and so on.

13.3.2 Child-level aggregations

We looked at categorizing data into date buckets in the last section. In addition to creating buckets with the respective ranges, we may want to aggregate data inside those buckets. For example, we may want to find the average rating of a book for each bucket.

To satisfy such requirements, we can use a *sub-aggregation*: an aggregation that works on the bucket's data. With bucketing aggregations, support for both metric and bucket aggregations is applied at the child level. The following listing fetches books released yearly plus the average rating for each bucket.

Listing 13.13 Average metric on books categorized per year

```
GET books/_search
{
  "size":0,
  "aggs": {
    "release_date_histogram": {
      "date_histogram": {
        "field": "release_date",
        "calendar_interval": "1y"
      },
      "aggs": {
        "avg_rating_per_bucket": {
          "avg": {
            "field": "amazon_rating"
          }
        }
      }
    }
  }
}
```

The bucketing histogram that categorizes books by year

Names the sub-aggregation

Applies a single-value metric across individual buckets

There are two blocks of aggregation, one woven inside the other. The outer aggregation (`release_date_histogram`) produces the data as a histogram based on the calendar interval of one year. The results of this aggregation are then fed to the next level of aggregation: the inner aggregation (`avg_rating_per_bucket`). The inner aggregation considers each bucket as its scope and runs the average (`avg`) aggregation on that